

Programación II

Desarrollo en Java

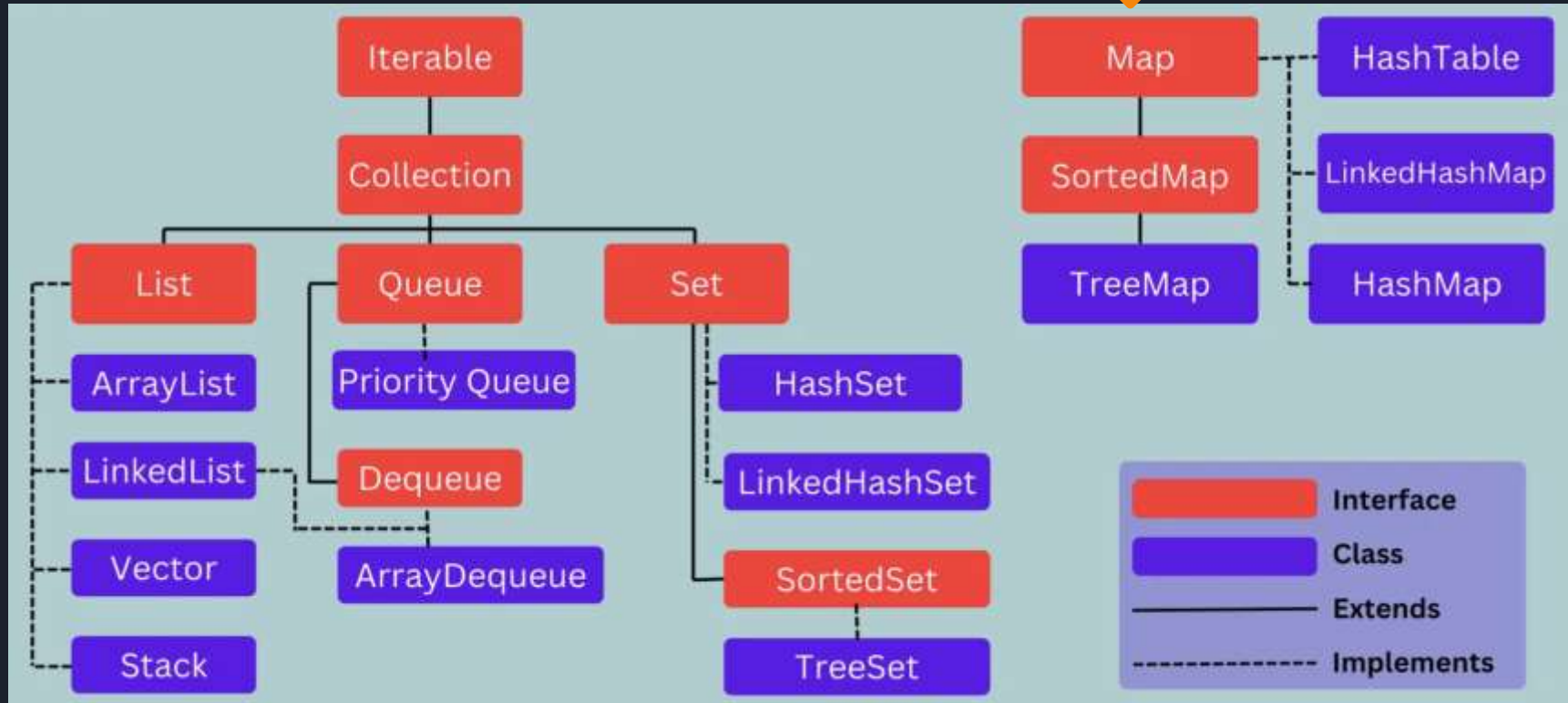
Clase N° 14:

Colecciones de tipo Map.

Profesora Carolina Archuby



INTERFAZ MAP



INTERFAZ MAP. Notas distintivas

=> Representa una estructura de datos que **ASOCIA** CLAVES **CON** VALORES.

=> La clave funciona como un identificador único (*conviene que sea inmutable -final-, de modo que no cambie en tiempo de ejecución*). **No se admiten claves duplicadas.**

=> **Para cada clave sólo habrá un valor, aunque diferentes claves pueden asociarse con el mismo valor.**

LAS INTERFACES MAP Y COLLECTION

=> Aunque muchas veces se hable de los mapas como una colección, en realidad no lo son, ya que **no heredan de la interfaz Collection** (debido a su estructura única de manejo de datos basada en pares clave-valor, que es diferente de las colecciones que manejan listas o conjuntos de valores). **Los mapas se definen en la interfaz Map.**

INTERFAZ MAP: Ventajas

=> Como los valores se insertan, recuperan o eliminan usando la clave correspondiente, **las operaciones de acceso son muy rápidas** dependiendo de la implementación del Map.

Interfaz Map: Métodos.

Map trabaja siempre en términos de clave-valor. Y si bien las claves no pueden estar repetidas, los valores sí.

=> Por eso **en Map sólo hay métodos implementados para buscar (get, remove) usando la clave**, no directamente por el valor. **Si queremos buscar por valor, tenemos que recorrer el mapa.**

Interfaz Map: Métodos.

- **Object put (K key, V value):** Inserta el par clave-valor en el mapa. Si la clave ya existe, el valor anterior se reemplaza por el valor especificado.
- **Object get (Object key):** Devuelve el valor asociado con la clave especificada, o null si el mapa no contiene la clave.
- **Object remove (Object key):** Elimina el par clave-valor asociado con la clave especificada y devuelve el valor al que la clave estaba previamente asociada (o null si no la encuentra).
- **Boolean remove (Object key, Object value):** Elimina la entrada solo si la clave está asociada exactamente a ese valor.

Interfaz Map: Métodos.

- **boolean containsKey (Object key):** Devuelve true si el mapa contiene la clave.
- **boolean containsValue (Object value):** Devuelve true si el mapa contiene una o más claves asociadas a este valor.
- **boolean isEmpty ():** Devuelve true si el Map no contiene ninguna entrada, y false en caso contrario.
- **int size ():** Devuelve la cantidad de pares clave-valor almacenados en el Map.

Interfaz Map: Forma de recorrer.

Para iterar sobre un Map, se pueden obtener vistas de:

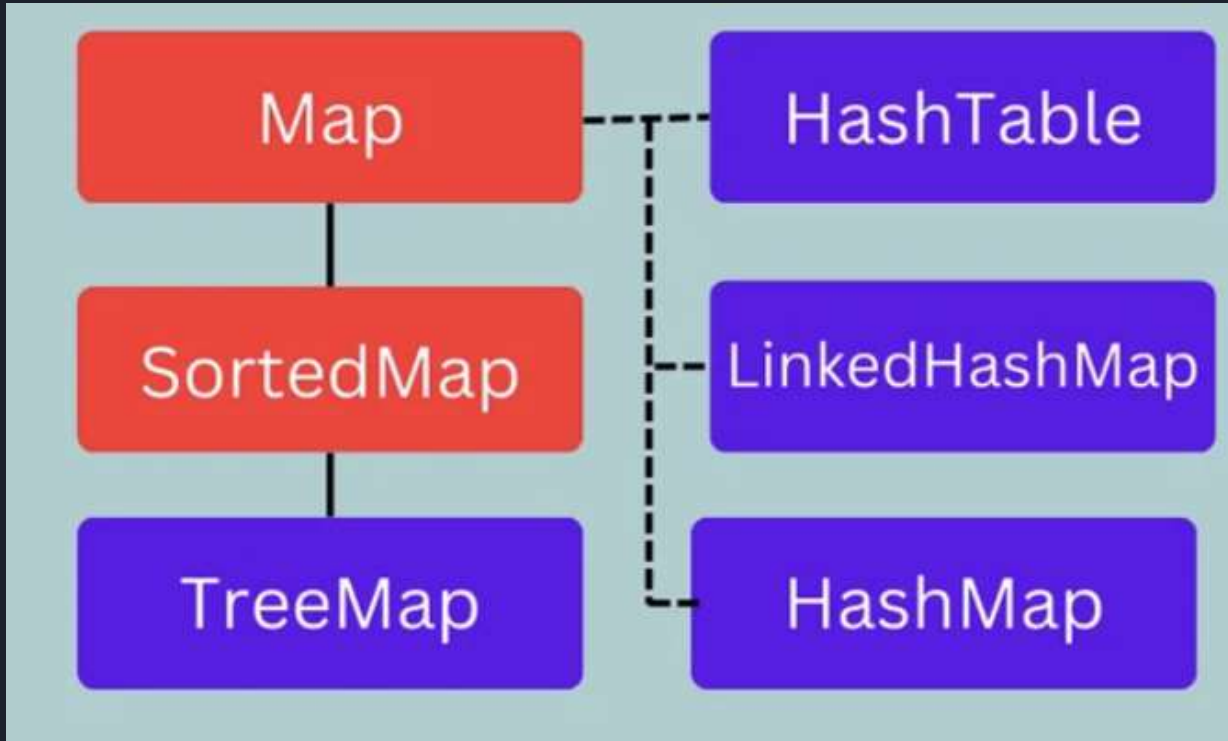
- las **entradas** => El método **entrySet ()** retorna un **Set** de las entradas (los pares clave-valor) contenidas en el mapa.
- las **claves** => El método **keySet ()** retorna un **Set** de las claves contenidas en el mapa.
- o los **valores** => **values ()** retorna una **Collection** de los valores contenidos en el mapa.

Esto permite utilizar todas las operaciones de iteración proporcionadas por esas interfaces

Iteración de las entradas:

```
Iterator it = hm.entrySet().iterator();  
while(it.hasNext())  
{  
    Map.Entry me = (Map.Entry) it.next();  
    System.out.println(me.getKey()+"-"+me.getValue());  
}
```

INTERFAZ MAP: CLASES



HASHMAP

=> LOS OBJETOS SE ALMACENAN UTILIZANDO UNA TABLA DE DISPERSIÓN (HASH), SIN ORDEN.

=> Hashing o dispersión: técnica de organización de registros que utiliza el código generado por el método hashCode. Otorga **alto rendimiento en operaciones básicas como inserción, eliminación y acceso**

=> Sólo se admite una clave null.

Operando con HashMap

```
// Inicializar un HashMap
    Map<String, Integer> miHashMap = new HashMap<> ();

// Agregar elementos
    miHashMap.put ("Uno", 1);
    miHashMap.put ("Dos", 2);
    miHashMap.put ("Tres", 3);

// Ver el tamaño del HashMap
    System.out.println ("Tamaño del HashMap: " + miHashMap.size () ) ;

// Eliminar un elemento
    miHashMap.remove ( key: "Dos" ) ;

// Verificar si contiene un elemento
    if (miHashMap.containsKey ("Uno") ) {
        System.out.println("El HashMap contiene la clave 'Uno'") ;
    }

// Recorrer el HashMap
    for (Map.Entry<String, Integer> entry: miHashMap.entrySet()){
        System.out.println("Clave: "+entry.getKey()+"", Valor: "+entry.getValue ());
    }
```

HASHTABLE

=> Es **SIMILAR A HashMap PERO SINCRONIZADO.**

=> Y además **no admite claves NULL.**

LINKEDHASHMAP

=> Similar a HashMap pero con la diferencia que mantiene una LISTA DOBLEMENTE VINCULADA, además de la tabla Hash.

=> Al usar una lista doblemente vinculada, los elementos quedan agregados en orden de inserción.

=> Permite iteraciones predecibles y ordenadas sin sacrificar el rendimiento significativo de HashMap.

Ejemplo de LINKEDHASHMAP

```
Map<String, String> diccionario = new LinkedHashMap<>();  
diccionario.put("brillante", "Que brilla.");  
diccionario.put ("efimero", "Que dura poco tiempo.");  
diccionario.put("sutil", "Delicado, tenue.");  
  
// Imprime las entradas en el orden en que fueron insertadas  
for (String key : diccionario.keySet()){  
    System.out.println(key + " : " + diccionario.get(key));  
}
```


TREEMAP



=> Utiliza un ÁRBOL BINARIO DE BUSQUEDA BALANCEADO (árbol rojo-negro).

=> Permite tener un mapa ORDENADO. La iteración será en orden ascendente según sus keys.

=> TreeMap **ordenará los valores de las claves en orden ascendente si usamos como key clases propias de Java (String, Integer, etc.). Si usamos como key clases creadas por el programador, habrá que implementar en ellas la interfaz Comparable.**

=> El coste de las operaciones básicas será logarítmico con el número de elementos del mapa $O(\log n)$.

TreeMap: Métodos específicos:

- => Object **ceilingKey()**: Retorna la **menor clave mayor o igual** a la clave pasada por parámetro. Si no existe ninguna clave mayor o igual, devuelve null
- => Object **firstKey()**: Devuelve la primera clave (**la más chica** en el orden).
- => Object **lastKey()**: Devuelve la última clave (**la más grande** en el orden).
- => Object **lowerKey(object key)**: Devuelve la **mayor clave estrictamente menor** a la pasada por parámetro, o null si no existe.
- => Object **higherKey(object key)**: Devuelve la **menor clave estrictamente mayor** a la pasada por parámetro, o null si no existe.

TreeMap. Ejemplo:

```
Map<String,Integer> libros = new TreeMap<>();  
libros.put("Cien años de soledad", 30);  
libros.put("Don Quijote", 15);  
libros.put ("Hamlet", 20);  
  
// Imprime las claves (que se encuentran en orden alfabético)  
for (Map.Entry<String, Integer> entrada : libros.entrySet()){  
    System.out.println(entrada.getKey()+" : "+entrada.getValue());  
}
```

HashMap vs. Hashtable vs TreeMap

=> La principal diferencia entre los tres es la **SINCRONIZACIÓN: Hashtable es el único sincronizado**, HashMap y TreeMap no. Por ello, para aplicaciones multihilos es preferible elegir Hashtable.

=> **HashMap es mejor en cuanto a performance.**

=> **TREEMAP ES EL ÚNICO ORDENADO**, HashMap y Hashtable no lo son.

=> **Hashtable no admite valores nulos** en ninguna de sus partes, mientras que HashMap permite tener una clave null y un valor null.

Resumen de clases que implementan Map:

HashMap

C

- No ordenación.
- Eficiente.

LinkedHashMap

C

- Ordenado por inserción.
- Permite ordenación por uso.
- Eficiente lectura.
- Poca eficiente escritura

TreeMap

C

- Ordenado por clave.
- Poco eficiente en todas sus operaciones

HashTable

C

- Considerado obsoleto.
- Utilizado en operaciones de concurrencia

Como resumen de todas las
colecciones:

Diagrama de decisiones
para el uso de colecciones:

INICIO

¿El orden es importante?

No

HashMap

Sí

¿Orden de inserción u orden de clave?

Ordenación

TreeMap

Inserción

LinkedHashMap

Pares

¿Tiene pares de clave/valor o sólo valores?

Valores

¿Contiene duplicados?

Sí

ArrayList

No

No

¿Su tarea principal es buscar elementos (contains/remove)?

Sí

¿El orden es importante?

No

HashSet

Sí

¿Orden de inserción u orden de elemento?

Inserción

LinkedHashSet

Ordenación

TreeSet